
ST2 Assignment 2- Group Project

- Group of 2 students (min) or 3 Students (max).
- Students need to form the group from same tutorial.
- You can work Solo/individual but there will no special consideration

Project Title: DSA Explorer and Visualiser App

1. Project Overview

Develop an interactive **Data Structures and Algorithms Explorer and Visualiser** that combines multiple algorithm visualizations with puzzle-like challenges. The app allows users to:

- Explore data structures and algorithms visually
- Interact with puzzles/games requiring application of learned algorithms and data structures
- See live animations of stacks, queues, linked lists, trees, graphs, heaps, sorting, and dynamic programming.
- Use [Pygame](#) package for visualisation

2. Key Requirements & Milestones/Deliverables

- The project needs to be modular with 3 key phases.
- The python modules developed in each module developed need to consist of a feature using a data structure and algorithm concept along with PyGame visualization and interactivity. Some of the broad requirements are outlined below:

1. Phase 1: Data Structure Playground (Aligned with Week 10 Assignment Tutorial)

- Visual stack and queue interaction with push/pop/enqueue/dequeue buttons.
- Linked list editor: insert/delete/reverse with pointer visualization.
- Binary Search Tree visualizer: insert nodes, display traversal orders (in/pre/post).

2. Phase 2: Algorithm Visualizer (Aligned with Week 11 Assignment Tutorial)

- Sorting algorithms visualizations: bubble, selection, merge sort with user-controlled start/reset.
- Graph traversal demo: BFS and DFS with user-selected start nodes on interactive graph.
- Heap visualizer: insertion/extraction shown step-by-step.

3. Phase 3: Puzzle Challenges (Integrated use of DS&A)

- **Pathfinding Puzzle:** User places start/end/obstacles on a grid; application runs Dijkstra's or A* to find shortest path dynamically.
- **Event Queue Simulator:** Simulate events scheduled by priority and time,

visualizing heap events processing.

- **Dynamic Programming Puzzle:** Grid path counting with obstacles and path reconstruction visualization.

Suggested App Layout

- Menu screen with buttons linking to the different modules: Data Structures, Sorting, Graphs, Heap, Puzzles.
- Each module has its own PyGame screen implementing the relevant visualizations and interactive controls.
- Common toolbar or instructions area explaining controls and algorithm status.

Implementation Suggestions

Step 1: Setup a main menu screen using PyGame.

- Provide clickable buttons that launch each module.
- Clean transition between modules.

Step 2: Build modular components for each module.

- Each component is a Python class or script file with isolated PyGame event loops/functions.
- Use consistent colors, fonts, and UI style.

Step 3: Implement interactivity and animations for each DS&A concept previously developed.

- Refactor code from tutorial class demos for reuse and enhancement.
- Add user input features (mouse, keyboard) and on-screen instructions.

Step 4: Combine puzzle challenges that require applying multiple algorithms.

- Example: Start with BFS/DFS to find paths, then visualize heap-based event processing.
- Use DP visualization to validate options or move counts in puzzles.

Step 5: Use Automated Testing (Python unit test module)

Sample Project Skeleton (Outline Code)

```
import pygame

import sys

pygame.init()
WIDTH, HEIGHT= 800, 600
screen= pygame.display.set_mode((WIDTH, HEIGHT))
FONT= pygame.font.SysFont(None, 36)
```

```

clock= pygame.time.Clock()

def draw_text(text, pos):
    txt = FONT.render(text, True, (0, 0, 0))
    screen.blit(txt, pos)

def main menu():
    screen.fill((200, 200, 250))
    draw_text("Algorithm Explorer", (WIDTH // 3, 50))
    buttons= {
        'Data Structures': pygame.Rect(300, 150, 200, 50),
        'Sorting': pygame.Rect(300, 230, 200, 50),
        'Graphs': pygame.Rect(300, 310, 200, 50),
        'Heap': pygame.Rect(300, 390, 200, 50),
        'Puzzles': pygame.Rect(300, 470, 200, 50)

    for text, rect in buttons.items():
        pygame.draw.rect(screen, (150, 150, 200), rect)
        draw_text(text, (rect.x + 20, rect.y + 10))
    pygame.display.flip()
    return buttons

# Placeholder functions for different modules
def data_structures module():
    # Implement stack, queue, linked list, BST visualization here
    pass

def sorting module():
    # Bubble sort, selection sort, merge sort visualizations
    pass

def graphs_module():
    # BFS, DFS visualization with interactive graph
    pass

def heap module():
    # Heap insertion and extraction visualization
    pass

def puzzles module():
    # Pathfinding, event simulation, DP puzzles
    pass

```

```

def main():
    running = True
    current module = None
    buttons = main_menu()

    while running:
        for event in pygame.event.get():
            if event.type== pygame.QUIT:
                running= False
            elif event.type== pygame.MOUSEBUTTONDOWN and
current module is None:
                pos = event.pos
                for name, rect in buttons.items():
                    if rect.collidepoint(pos):
                        current module= name

        if current module is None:
            buttons= main_menu()
        else:
            if current module== 'Data Structures':
                data_structures module()
            elif current module== 'Sorting':
                sorting_module()
            elif current module == 'Graphs':
                graphs_module()
            elif current module == 'Heap':
                heap_module()
            elif current module == 'Puzzles':
                puzzles_module()

            # For demo, after module ends return to menu
            current module= None

        clock.tick(30)

    pygame.quit()

if __name__ == "__main__":
    main()

```

Submission Requirements

- Submit a project report with each phase implementation and testing screenshot and full source code for DSA Explorer and Visualiser through GitHub, with at least **3 modules fully implemented and tested**, each demonstrating at least 2 algorithm/data structure concepts with PyGame visualization.
- Demonstrate the fully working app in the tutorial class in week 13 tutorial/lab before submission to GitHub

Automated Test Cases and Validation Scenarios

Here are few test case descriptions for automated testing as samples/examples

Module 1: Data Structures Test Plan

Test Case	Description	Expected Result
Stack push/pop sequence	Push 3 items, pop 2	Final stack size = 1, correct order
Queue enqueue/dequeue	Enqueue 4 items, dequeue 3	FIFO order maintained
Linked list insertion at pos	Insert node with value 10 at position 2	Node present in correct location in list
BST insert and inorder traversal	Insert [50,30,70]; inorder traversal	Traversal order: 30, 50, 70

Module 2: Sorting Test Plan

Test Case	Description	Expected Result
Bubble sort correctness	Sort array of [5,3,8,1,2]	Sorted array: [1,2,3,5,8]
Selection sort animation	Ensure colors highlight compared elements	Highlight changes each step correctly

Module 3: Graph Traversal Test Plan

Test Case	Description	Expected Result
BFS traversal from 'A'	Visit nodes reachable from 'A'	Nodes visited in correct BFS order
DFS traversal from 'C'	DFS order matches expected path	Traversal path matches theoretical DFS
Interactive start node	Clicking different nodes	BFS restarts correctly from chosen node

Grading/Marking Rubric for Assignment 2- Group Project

Total: 30 points (Please Check Canvas for better visuals of the marking rubric)

Criteria	Ratings				Pts
DSA Algorithm Explorer Implementation Code Phase 1: 2 marks Phase 2 : 2 marks Phase 3: 2 marks	6 Pts Excellent - Exceeds Expectation Correct and Efficient Implementation of all algorithms for all three phases	4 Pts Good: Meets Expectations Mostly Correct and minor inefficacies or missing features in each of the phases	2 Pts Satisfactory: Progressing Towards Expectations Basic Implementation: covers key algorithms in each phase, some errors	1 Pts Needs Improvement : Clearly Below Expectations Multiple errors or incomplete algorithms in each phase	6 pts
Pygame Visualisation Implementation Code Phase 1: 2 marks Phase 2 : 2 marks Phase 3: 2 marks	6 Pts Excellent - Exceeds Expectation Clear, interactive visualizations with smooth animations for all algorithms in each phase	4 Pts Good: Meets Expectations Visualization present with some interactivity and minor glitches in each phase	2 Pts Satisfactory: Progressing Towards Expectations Basic Visuals, limited interactivity, minimal animations	1 Pts Needs Improvement : Clearly Below Expectations Poor or missing visualization, no interactivity	6 pts
Automated Testing and Benchmarking with Python Unit Test Package Phase 1: 2 marks Phase 2 : 2 marks Phase 3: 2 marks	6 Pts Excellent - Exceeds Expectation Implemented automated testing and benchmarking for all three phases	4 Pts Good: Meets Expectations Partial automated testing implementation done (only two phases)	2 Pts Satisfactory: Progressing Towards Expectations Partial automated testing and benchmarking (just 1 phase)	1 Pts Needs Improvement : Clearly Below Expectations Poor or missing automated testing with Python Unit Test Package or no benchmarking tests done	6 pts
Non function requirements: User Interaction, integration and Navigation and Code Quality and Modularity User Interaction, Integration and Navigation Phase 1: 1 mark Phase 2 : 1 mark Phase 3: 1 mark Code Quality and Modularity Phase 1: (1 mark) Phase 2: (1 mark) Phase 3: (1 mark)	6 Pts Excellent - Exceeds Expectation Non-functional requirements: User Interaction: Intuitive Controls (mouse/keyboard) and well documented for each phase. Integration and Navigation: Seamless module switching with main menu and exit Code Quality and Modularity: Clean, modular, well commented code structure for each phase	4 Pts Good: Meets Expectations Non functional Requirements: User Interaction: Controls available, somewhat intuitive for each phase. Integration and Navigation: Menu Functional with minor UI issues. Code Quality and Modularity: Somewhat organized and minimal comments in each phase	2 Pts Satisfactory: Progressing Towards Expectations Nonfunctional requirements: User Interaction: Controls basic and unclear in each phase. Integration and Navigation: Basic Menu Implemented. Code Quality and Modularity: somewhat organized with minimal comments in each phase. phase	1 Pts Needs Improvement : Clearly Below Expectations Non-functional Requirements: User Interaction: No user interaction or very hard to use for each phase. Integration and Navigation: No main menu or poor navigation between modules. Code Quality and Modularity: Poorly organized, hard to understand for each phase.	6 pts
Demo and Submission/Deliverables: Successful demonstration in tutorial class and timely submission of deliverables In-class demonstration of complete working app in week 13 tutorial class (3 marks) Timely final submission to Git-Hub in week 13 (3 marks)	6 Pts Excellent - Exceeds Expectation Successful Demonstration of phase 1, phase 2 and phase 3 requirements done in week 13 tutorial. Successful submission of phase 1, phase 2 and phase 3 reports to github and github link submission in Canvas Assignment submission in Week 13.	4 Pts Good: Meets Expectations Successful Demonstration of only two phases out of 3 phases requirements done in week 13 tutorial. Successful submission of reports to GitHub, but no github link provided in Canvas Assignment submission in Week 13.	2 Pts Satisfactory: Progressing Towards Expectations Successful Demonstration of only one phases out of 3 phases requirements done in week 13 tutorial. No time submission of reports to GitHub as evident from timestamp, or no github link provided in Canvas Assignment submission in Week 13.	1 Pts Needs Improvement : Clearly Below Expectations No Demonstration or partially working demo of any one phase out of 3 phases requirements done in week 13 tutorial. No submission of reports to GitHub, or no github link provided in Canvas Assignment submission in Week 13.	6 pts
Total points: 30					